

Fundamentos da Qualidade e Produtividade

02

2026

SENAI - CRTI

Ferramentas da Qualidade e Filosofia Lean

Unidade Curricular: **Fundamentos da Qualidade e Produtividade**



Agenda da Aula

O que vamos construir juntos hoje?

01

Situação-Problema

Contextualização com um cenário real de crise em projeto de software

02

Mobilização

Reflexão sobre experiências e conhecimentos prévios da turma

03

Conteúdo Teórico

MAASP, Fluxogramas, Diagrama de Ishikawa e Filosofia Lean

04

Prática Guiada

Aplicação das ferramentas em cenários de TI

05

Sistematização e Fechamento

Resumo visual e retomada da situação-problema

Capacidades a Desenvolver

O que você será capaz de fazer ao final desta aula



Analisar Problemas

Aplicar ferramentas da qualidade para análise e resolução estruturada de problemas em TI



Pensar Lean

Compreender e aplicar os princípios da filosofia Lean em processos de desenvolvimento de software



Identificar Desperdícios

Enxergar e eliminar desperdícios ocultos em fluxos de trabalho de desenvolvimento



Visão Sistêmica

Desenvolver visão macro e micro sobre os processos organizacionais e suas interdependências

● Situação-Problema

O Projeto que Não Para de Atrasar

Uma equipe de desenvolvimento enfrenta **atrasos constantes**, retrabalho crescente e estouro de custos. O cliente está insatisfeito, o time está sobrecarregado e ninguém consegue identificar a causa raiz do problema. Parece familiar?



O Cenário em Detalhes

O que está acontecendo?

- Bugs recorrentes chegando em produção
- Retrabalho consumindo até 40% do tempo da equipe
- Entregas atrasadas em todas as sprints
- Custos operacionais acima do orçamento
- Comunicação falha entre frontend, backend e QA

O que está faltando?

A equipe não possui **análise estruturada de problemas** nem metodologia para identificar e eliminar as causas raiz.

Os processos são informais, sem documentação e repletos de **desperdícios invisíveis** que ninguém ainda mapeou.

- Este é exatamente o tipo de problema que as ferramentas da qualidade e o Lean foram criados para resolver.

💬 Mobilização de Conhecimentos

Antes de começar — o que você já sabe?

Vamos refletir juntos. Não existe resposta errada — queremos ativar o que você já vivenciou como desenvolvedor ou como usuário de sistemas.



Perguntas para Reflexão

🧐 Por que projetos atrasam mesmo com planejamento?

Você já fez parte de um projeto com cronograma definido que mesmo assim atrasou? O que aconteceu? Quais foram as causas?

🔄 O que gera retrabalho em desenvolvimento de software?

Já precisou refazer uma funcionalidade do zero? O que levou a isso — requisitos mal definidos, falta de testes, comunicação falha?

⚙️ Já viu processos confusos ou ineficientes?

Alguma vez você percebeu que uma tarefa poderia ser feita de forma muito mais simples, mas era obrigatório seguir uma série de etapas desnecessárias?

Essas experiências são o ponto de partida para entender **por que precisamos de ferramentas estruturadas de qualidade.**

Métodos e Ferramentas da Qualidade

Conheça as principais ferramentas utilizadas para análise, visualização e resolução de problemas em processos de TI.

MASP

Método de Análise e Solução de Problemas — estrutura lógica em 8 etapas

Fluxogramas

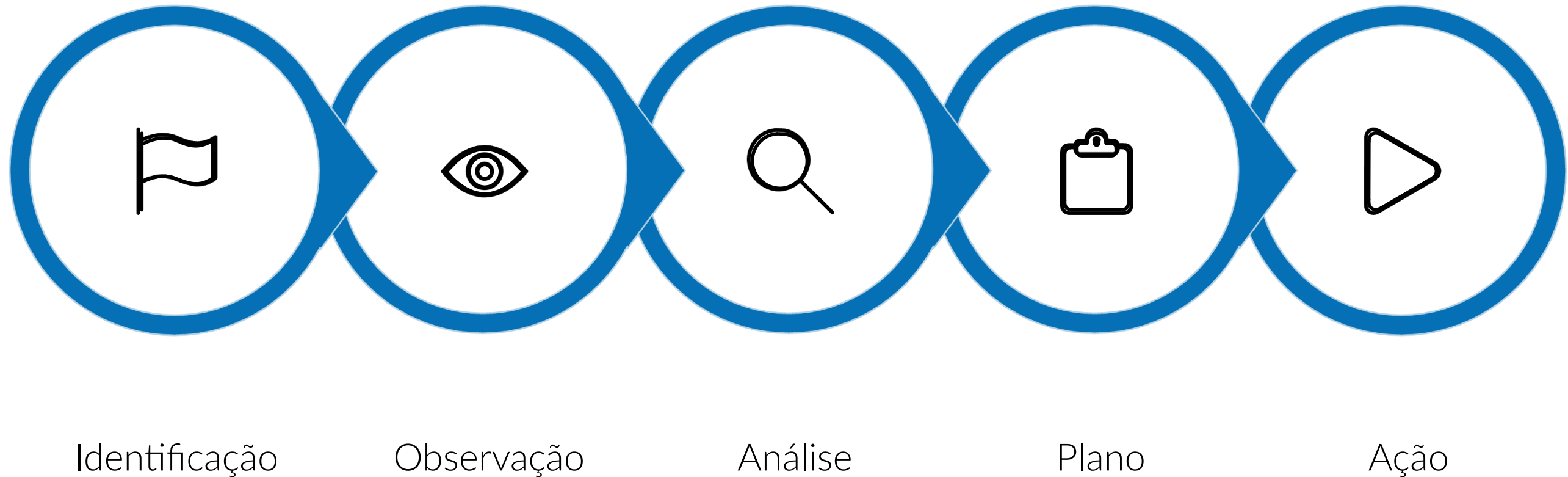
Representação visual de fluxos e processos — identifica gargalos e etapas desnecessárias

Diagrama de Ishikawa

Diagrama de causa e efeito — mapeamento das causas raiz de um problema

MASP — Método de Análise e Solução de Problemas

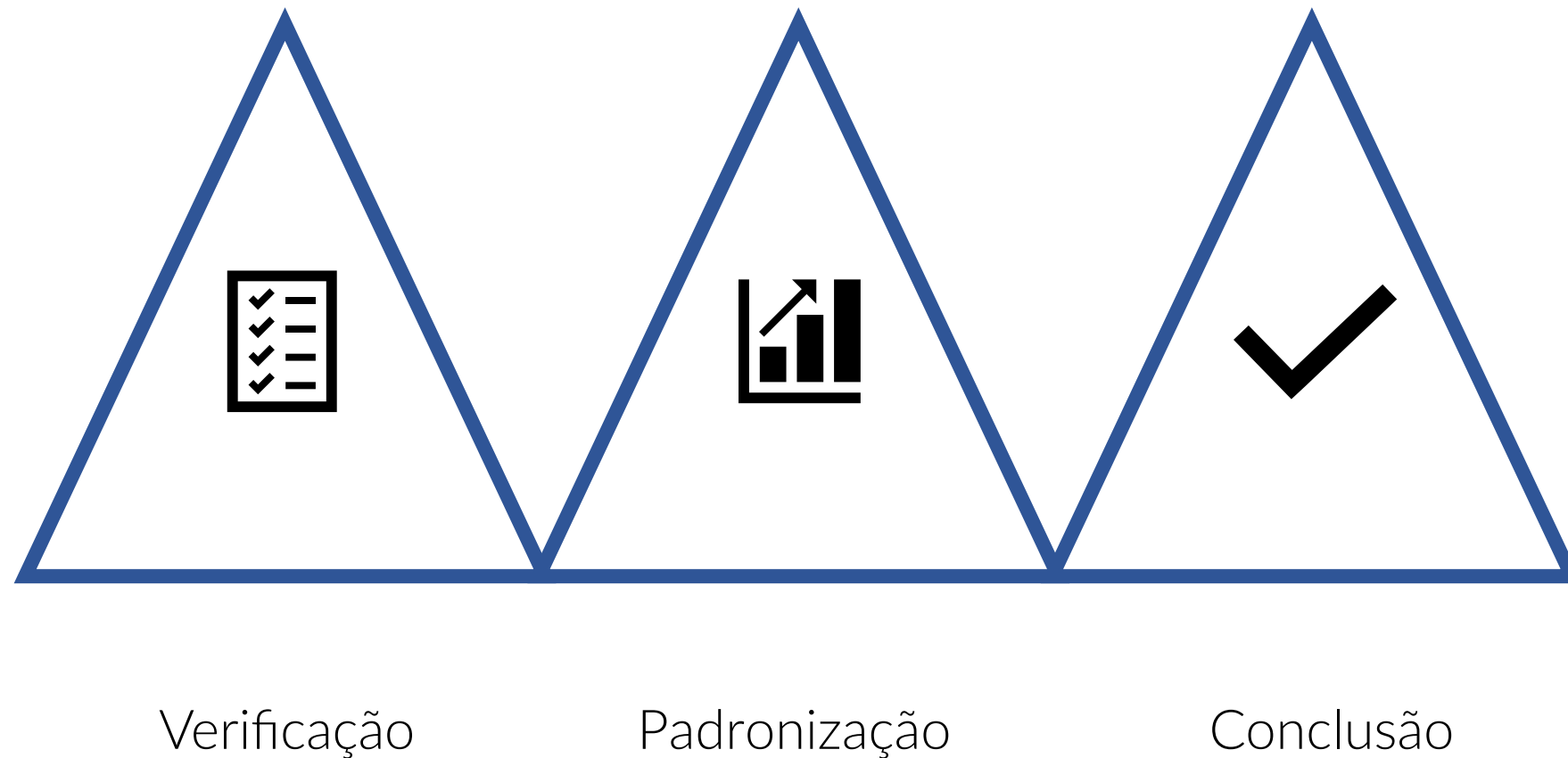
Uma abordagem estruturada em 8 etapas



O MASP é baseado no ciclo PDCA e garante que problemas não sejam resolvidos de forma superficial, mas com **investigação profunda das causas raiz** antes de qualquer ação.

MASP — Método de Análise e Solução de Problemas

Uma abordagem estruturada em 8 etapas



O MASP é baseado no ciclo PDCA e garante que problemas não sejam resolvidos de forma superficial, mas com **investigação profunda das causas raiz** antes de qualquer ação.

Como usar o MASP para resolver bugs e falhas de sistema

1

Identificar o Problema

Exemplo: "O sistema de login está falhando intermitentemente em produção para 15% dos usuários."

2

Observar e Coletar Dados

Analisar logs, rastrear a frequência e o contexto das falhas — quando ocorre, em qual browser, em qual horário.

3

Analisar as Causas

Usar o Diagrama de Ishikawa para mapear causas possíveis: sessão expirada, token inválido, falha de rede.

4

Planejar, Executar e Verificar

Criar um plano de correção, implementar, testar em ambiente de homologação e validar com dados reais.



Segundo o Ciclo PDCA:

■ Plan ■ Do ■ Check ■ Act

❏ O MASP evita o "apagar incêndio" — soluções rápidas que não resolvem o problema de verdade e fazem o bug voltar.

Fluxogramas

Tornando processos visíveis para torná-los melhoráveis

Um fluxograma é a representação gráfica de um processo. No desenvolvimento de software, ele permite enxergar **onde estão os gargalos, as decisões críticas e as etapas redundantes** antes mesmo de começar a codificar.



Losango — Decisão

Representa um ponto de decisão no fluxo (Sim/Não). Exemplo: "O teste passou?" → Sim: deploy / Não: corrigir.



Retângulo — Processo

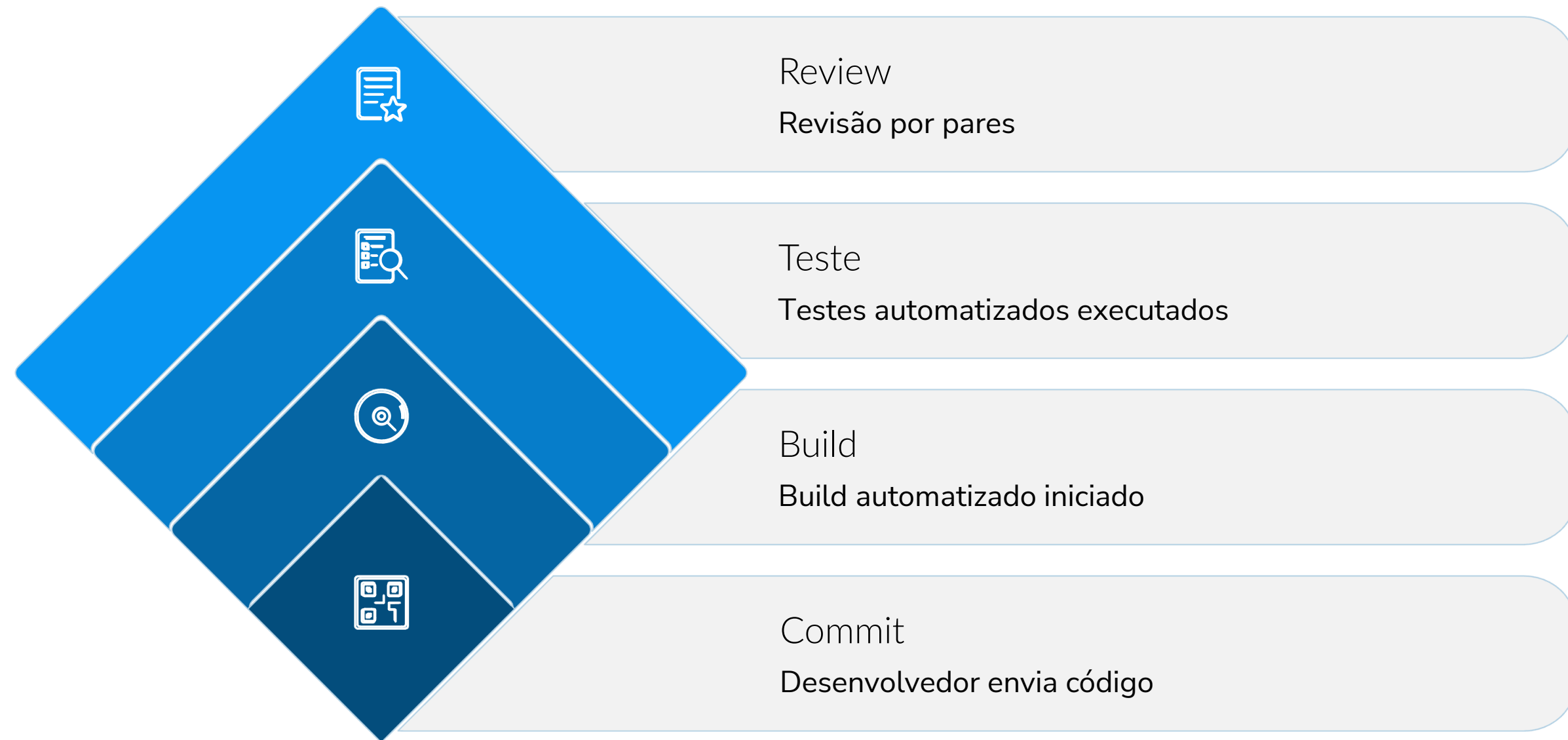
Representa uma ação ou atividade realizada. Exemplo: "Executar suite de testes automatizados".



Oval — Início/Fim

Marca o início e o fim do fluxo. Exemplo: "Início: Commit realizado" → "Fim: Feature em produção".

Fluxo de desenvolvimento: Commit → Teste → Deploy



Este fluxo representa um pipeline moderno de CI/CD. Identificar cada etapa visualmente permite detectar **onde o processo trava** com mais frequência — seja no review, nos testes ou no deploy.

Construa seu próprio fluxograma

Desafio para a turma

Em duplas ou trios, mapeie o fluxo de uma das seguintes situações

- Processo de correção de um bug reportado pelo cliente
- Fluxo de uma pull request — do código ao merge
- Processo de onboarding de um novo desenvolvedor na equipe

Como fazer

1. Identifique o **início** e o **fim** do processo
2. Liste todas as etapas em ordem
3. Identifique os pontos de **decisão** (Sim/Não)
4. Desenhe o fluxo usando os símbolos corretos
5. Identifique onde podem estar os **gargalos**

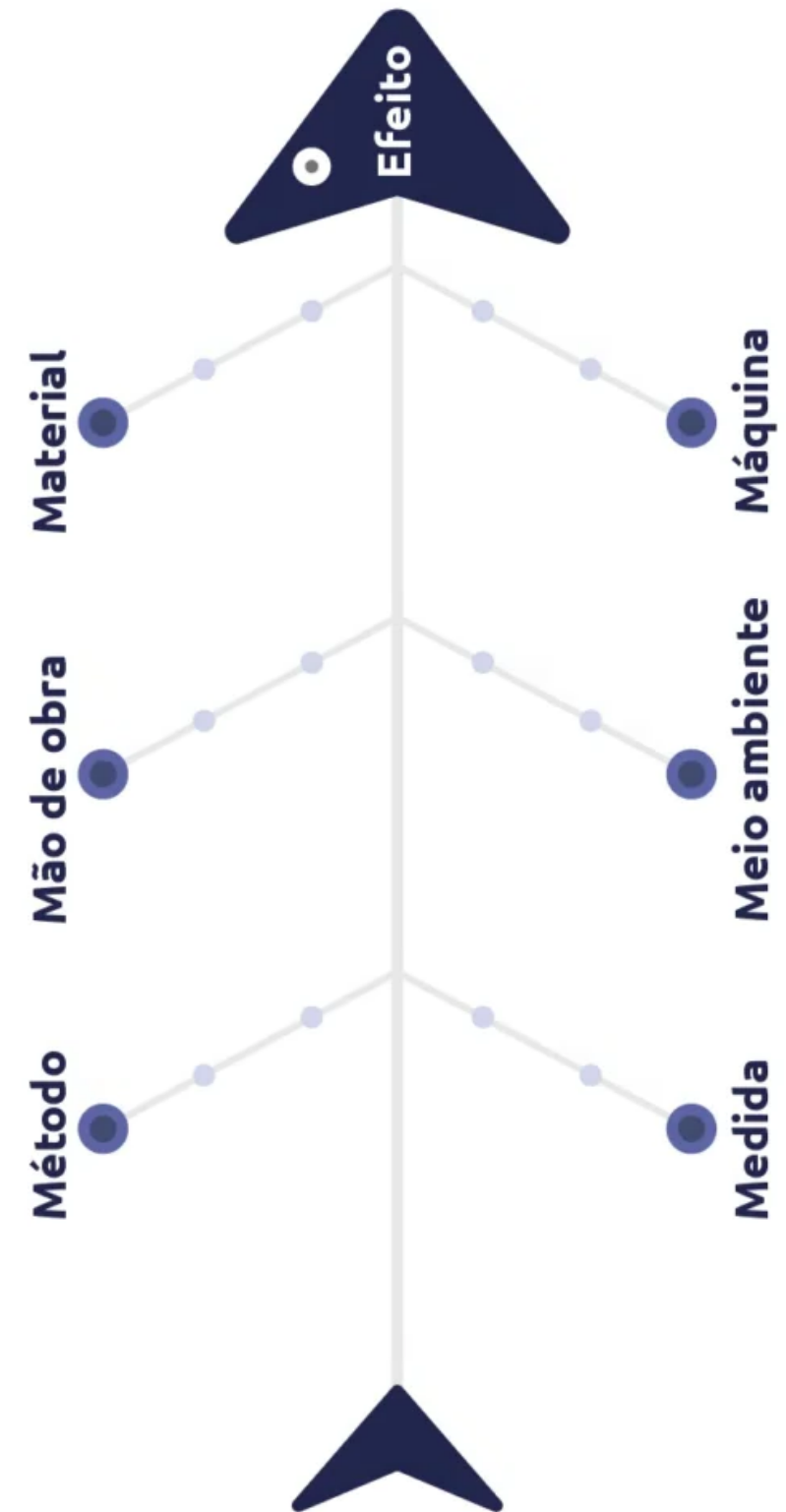


 Tempo: 10 minutos. Em seguida, 2 grupos apresentam seu fluxograma para a turma.

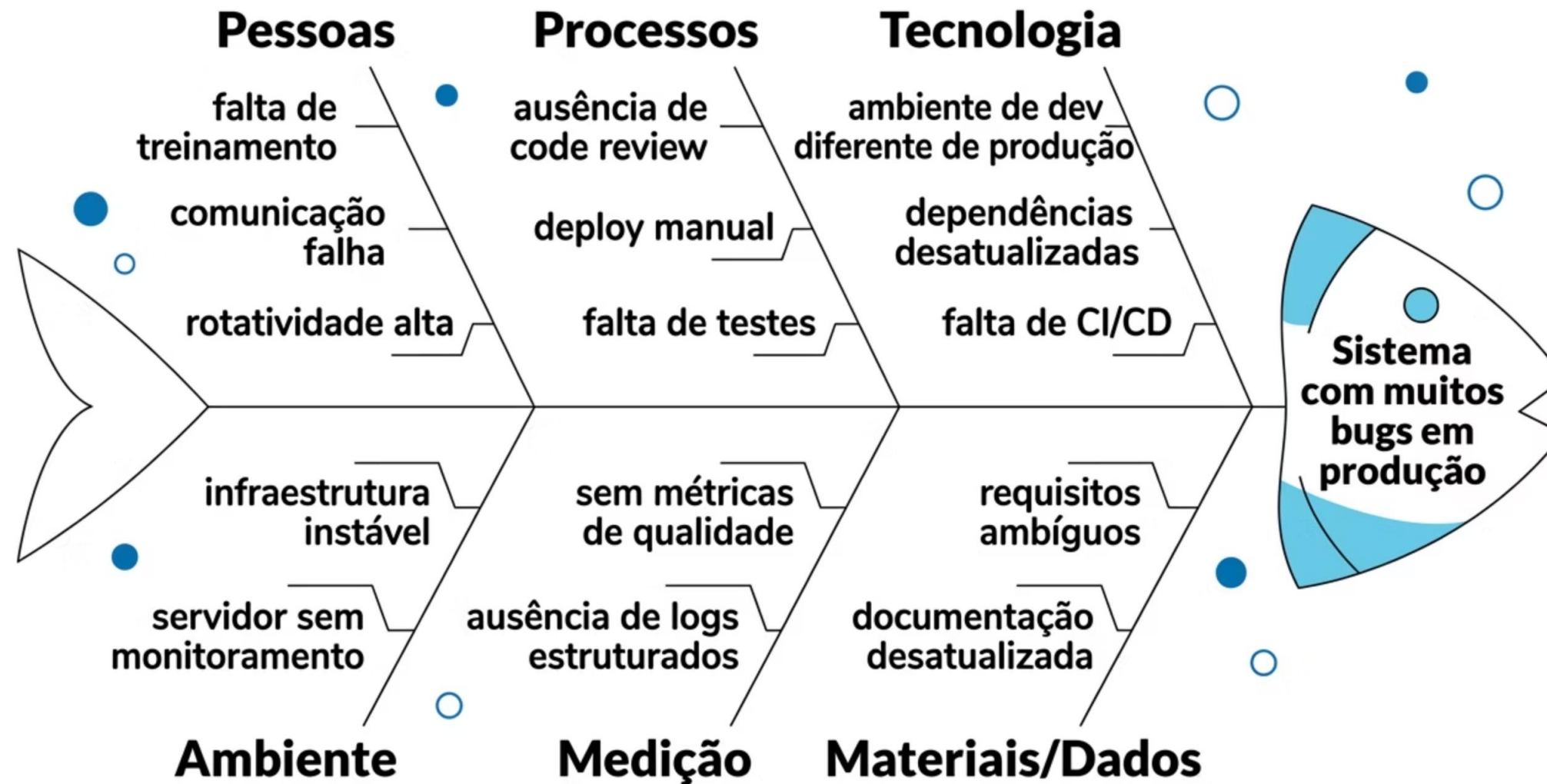
Diagrama de Ishikawa

Encontrando a raiz do problema — não apenas os sintomas

O Diagrama de Ishikawa (ou Espinha de Peixe) é uma ferramenta visual que organiza as **possíveis causas de um problema** em categorias, evitando que a equipe trate apenas os sintomas superficiais.



As 6 categorias de causa aplicadas à TI



Cada categoria revela um **domínio de investigação**. Ao preencher o diagrama em equipe, evita-se o viés de "a culpa é do código" e abre-se espaço para análises mais completas.

Crie um Diagrama de Ishikawa

Problema a analisar

"O sistema está apresentando erros frequentes após cada atualização de versão."

Este é um problema clássico em equipes que não possuem processos de release bem definidos.

Instruções

1. Desenhe a espinha de peixe com o problema no "cabeçote"
2. Identifique ao menos **3 categorias** de causa (Pessoas, Processo, Tecnologia...)
3. Para cada categoria, liste de 2 a 3 possíveis causas
4. Discuta com o grupo: qual causa parece ser a **raiz principal**?



Tempo: 12 minutos. Compartilhe com a turma as causas mais votadas como raiz.



CAPÍTULO 2

Filosofia Lean

Originada na Toyota, a filosofia Lean chegou ao desenvolvimento de software trazendo um princípio central: **entregar mais valor com menos desperdício.**

O que é valor? Só o cliente pode definir

Definição de Valor

Valor é tudo aquilo pelo qual o cliente **estaria disposto a pagar**. Qualquer atividade que não contribui diretamente para entregar esse valor é, por definição, um desperdício.

Em desenvolvimento de software: o cliente paga por **funcionalidades que funcionam**, não por reuniões, retrabalho ou burocracia interna.

Exemplos na prática

- **Tem valor:** nova feature entregue e testada, bug corrigido, melhoria de performance perceptível ao usuário
- **Sem valor (mas necessário):** documentação mínima, reunião de alinhamento curta
- **Desperdício puro:** reunião de status de 2 horas, reescrever código por falta de requisitos claros, aguardar aprovação manual desnecessária

Identificando o que não agrega valor no desenvolvimento



Reconhecer esses desperdícios no dia a dia é o primeiro passo para eliminá-los. Muitos são invisíveis porque já se tornaram parte da rotina da equipe.

Caça ao Desperdício — identifique os vilões do processo

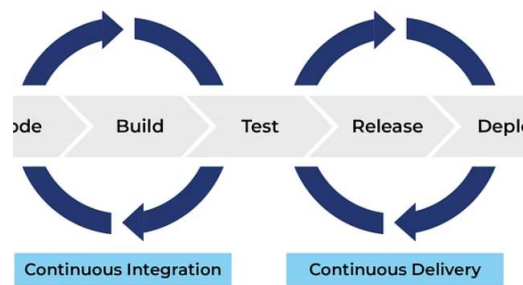
Leia o cenário abaixo e identifique **quantos e quais tipos de desperdício** você consegue encontrar:

"A equipe termina o desenvolvimento de uma feature na sexta à tarde. O código fica aguardando revisão porque o tech lead está em reuniões na segunda. Na terça, durante o code review, são encontrados 12 erros de padronização que poderiam ter sido detectados por um linter automático. O desenvolvedor corrige na quarta. O deploy é feito manualmente na quinta pelo DevOps. Na sexta seguinte, o cliente reporta que a feature não funciona no mobile — algo que não foi testado."

  Tempo: 8 minutos em grupo. Identifique cada desperdício, nomeie-o (espera, retrabalho, defeito...) e proponha uma solução Lean.

Lean no Desenvolvimento de Software

Práticas Lean que você já conhece (mas talvez não saiba)



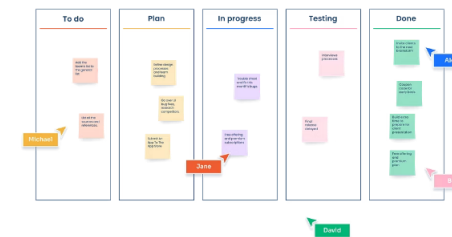
Integração Contínua (CI/CD)

Automatizar o processo de build e deploy elimina o desperdício de **espera** e **erros de integração manual**. Cada commit é validado automaticamente.



Automação de Testes

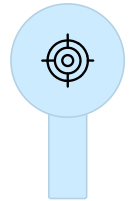
Testes automatizados eliminam o retrabalho causado por **defeitos descobertos tarde demais**. Quanto mais cedo o bug é encontrado, mais barato é corrigi-lo.



Kanban e Fluxo Puxado

Limitar o trabalho em progresso (WIP) reduz o desperdício de **multitarefa excessiva** e torna os gargalos visíveis para toda a equipe.

Os 5 princípios do Lean aplicados ao desenvolvimento



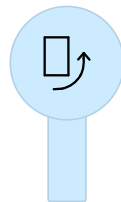
1. Definir Valor

Entender o que o cliente realmente precisa — não o que a equipe acha que precisa ser entregue.



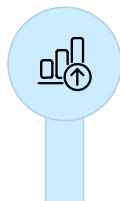
2. Mapear o Fluxo de Valor

Visualizar todo o processo de entrega para identificar onde há desperdícios e onde há valor agregado.



3. Criar Fluxo Contínuo

Eliminar interrupções, filas e esperas para que as funcionalidades fluam suavemente do desenvolvimento à produção.



4. Produção Puxada

Desenvolver o que o cliente solicita, quando solicita — evitando superprodução de features não utilizadas.



5. Buscar a Perfeição

Melhorar continuamente os processos — o Lean não é um projeto com fim, é uma cultura permanente de melhoria.

CAPÍTULO 3

Visão Sistêmica

Uma falha local raramente é um problema isolado. Entender como as partes de um sistema se conectam é essencial para entregar software com qualidade.



Micro e Macro — dois olhares necessários sobre os processos

Visão Micro

Nível da tarefa ou atividade

Foco no detalhe: Como essa função está codificada? Este teste cobre os casos extremos? Esta query está otimizada?

A visão micro garante **qualidade técnica** no nível do código e da atividade individual.

Visão Macro

Nível do processo organizacional

Foco no todo: Como o fluxo de desenvolvimento está integrado? Onde a equipe perde mais tempo? Como uma entrega impacta outra equipe?

A visão macro garante que o trabalho individual **serve ao objetivo maior** — entregar valor ao cliente.

 O desenvolvedor excelente transita entre os dois níveis: é preciso no detalhe e consciente do impacto sistêmico das suas decisões.

Como uma falha local se propaga pelo sistema



Este encadeamento mostra que a decisão de "**pular os testes por falta de tempo**" gera um custo muito maior do que o tempo economizado. A visão sistêmica nos ajuda a tomar decisões locais mais inteligentes.

O impacto da falta de comunicação entre times

Frontend ↔ Backend

Quando não há contrato de API claro, o frontend é desenvolvido com base em suposições. Resultado: **retrabalho em ambos os lados** quando as interfaces não se encaixam.

Dev ↔ QA

Quando o QA entra apenas no final do processo, os bugs encontrados exigem que o desenvolvedor retorne ao contexto de um código que já "virou página". **Custo alto, entrega lenta.**

Dev ↔ DevOps

Quando o deploy é responsabilidade exclusiva do DevOps sem automação, cria-se um **gargalo humano**: toda entrega depende da disponibilidade de uma única pessoa.

Equipe ↔ Cliente

Sem feedback contínuo do cliente, a equipe pode passar semanas construindo funcionalidades que não correspondem às necessidades reais — **desperdício de superprodução** clássico do Lean.

Quando usar cada ferramenta?

Ferramenta	Quando usar	Exemplo em TI
MAASP	Quando há um problema recorrente e precisa de análise estruturada antes de agir	Bug que retorna após cada correção superficial
Fluxograma	Quando precisar mapear, documentar ou melhorar um processo	Pipeline de CI/CD, fluxo de atendimento a incidentes
Ishikawa	Quando precisar investigar causas de um problema de forma colaborativa	Sistema instável em produção após deploy
Lean / Mapa de Valor	Quando quiser otimizar o fluxo de entrega e eliminar desperdícios	Sprint com muitas tarefas mas pouco progresso real

Sistematização — Resumo Visual

Os Princípios do Lean em uma visão

Definir Valor
O cliente define o que tem valor — tudo
o mais é candidato a eliminação

Melhorar Sempre
A perfeição é uma jornada — cada
processo pode sempre ser melhorado



Mapear Fluxo

Visualizar todo o caminho do valor — do
requisito à entrega em produção

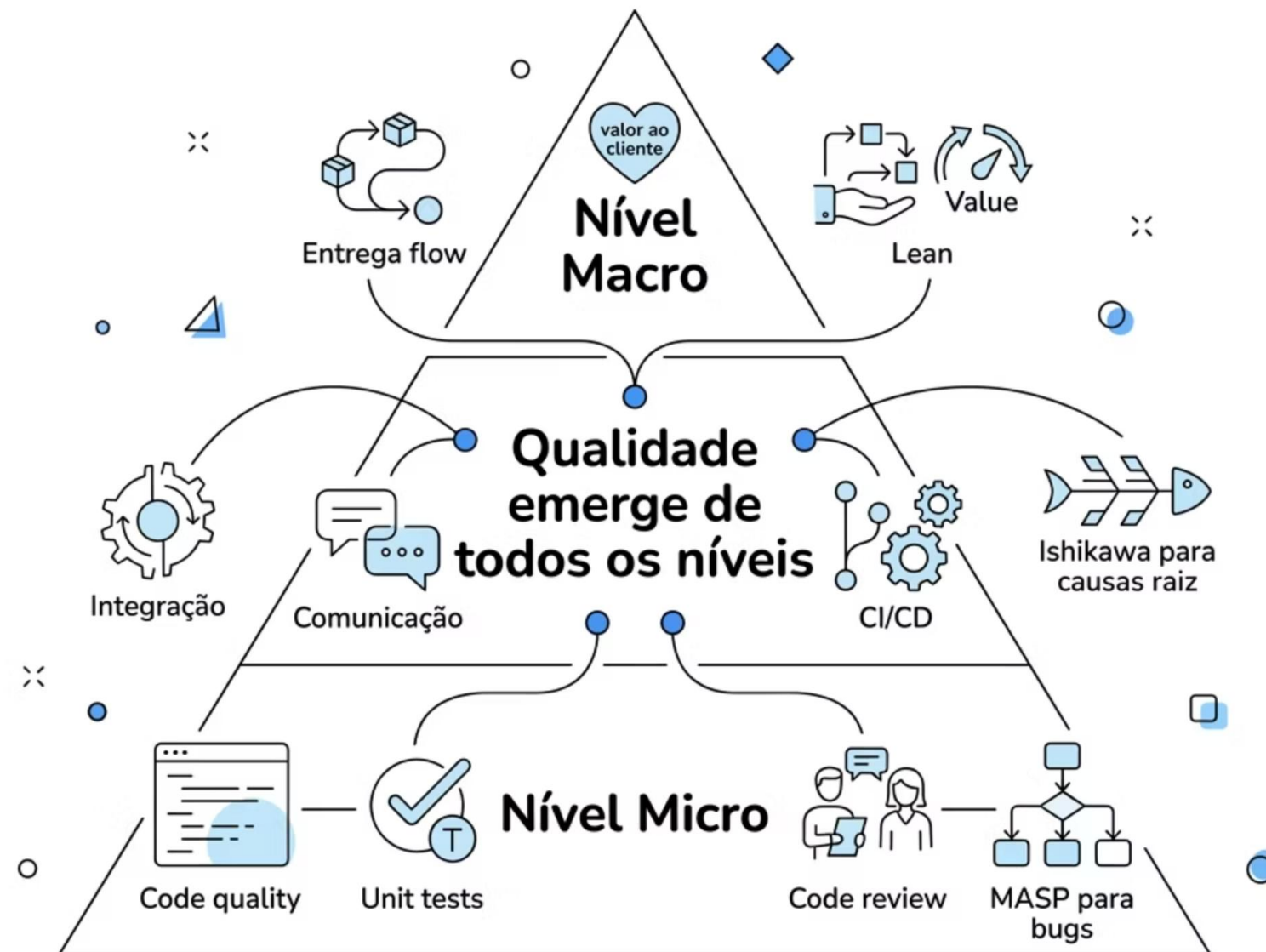
Criar Fluxo

Remover obstáculos para que as
entregas fluam sem interrupções

Puxar

Só produzir o que o cliente realmente
demanda — evitar superprodução

A relação entre olhar o todo e entregar qualidade



Qualidade em software não é apenas "código sem bugs". É o resultado de decisões conscientes em **todos os níveis** — do desenvolvedor individual ao processo organizacional.

● Fechamento — Retomada da Situação-Problema

Voltando ao início: como resolver o problema da equipe?

Lembra da equipe com atrasos, retrabalho e custos estourados? Agora temos as ferramentas para ajudá-la.



Como as ferramentas resolveriam o cenário real?

MASP para os bugs recorrentes

Em vez de "apagar incêndios", a equipe aplica as 8 etapas do MASP: identifica o problema com dados, analisa causas com Ishikawa, define ações e padroniza a solução para que o bug não volte.

Fluxograma para mapear o processo

Ao visualizar o fluxo completo — do commit ao deploy — a equipe identifica que o gargalo está na revisão manual, que leva em média 4 dias. A solução: processo de review estruturado com SLA definido.

Lean para eliminar desperdícios

A equipe mapeia os 7 desperdícios no processo atual, prioriza a automação dos testes e do deploy, elimina reuniões de status desnecessárias e adota Kanban para limitar o WIP.

Reflexão Final

A melhoria contínua não é um destino — é uma cultura

O que muda com essas ferramentas?

Ferramentas da qualidade e Lean não são soluções mágicas. São **formas de pensar** que, quando adotadas pela equipe, transformam a maneira de trabalhar permanentemente.

A equipe que analisa antes de agir, que enxerga o processo completo e que questiona constantemente "isso agrega valor?" — essa equipe entrega software de qualidade de forma sustentável.

Kaizen — Pequenas melhorias, grande impacto

- Comece mapeando **um processo** da sua equipe com fluxograma
- Na próxima retrospectiva, identifique **um desperdício** e proponha eliminá-lo
- Diante do próximo bug recorrente, aplique **ao menos as 4 primeiras etapas do MASP**
- Pergunte sempre: "**Isso agrega valor ao cliente?**"

